Stable Diffusion

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

Table of Contents

What is Stable Diffusion?

- The model that made high-quality text-to-image generation openly available (2022).
- Built by the CompVis group (Ommer lab) with Runway, trained on a compute donation from Stability AI.
- Open weights, which is what enabled the wide adoption and customization that followed.

Text-to-image samples from Stable Diffusion. Source: CompVis, stable-diffusion repository README (`merged-0007.png`).

Stable Diffusion (cont.)

Advantages

- High-quality, diverse image generation.
- Efficient training and inference due to latent space operations.
- Open weights, so the model can be inspected, fine-tuned and redistributed.

Stable Diffusion (cont.)

What do we need to make a good generative model?

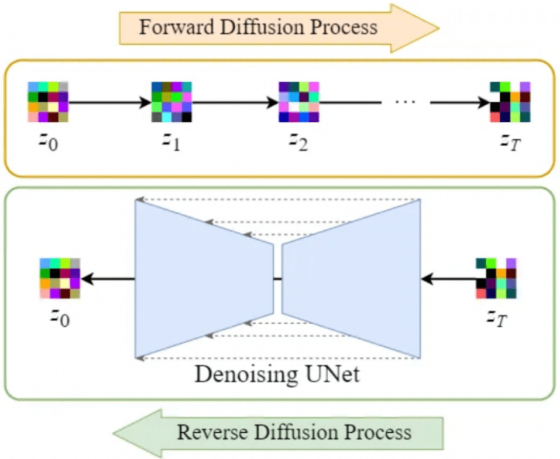
1. Method of learning to generate new images	**Forward/reverse diffusion**
2. Way to link text and images	**Text-image representation model**
3. Way to compress images	**Autoencoder**
4. Way to add in good inductive biases	**U-net architecture + 'attention'**

Making a 'good' generative model is about making all these parts work together well!

Generative Modeling

Where We Left Off

- GANs and VAEs made image synthesis work, but each pays for it: GANs with unstable training and mode collapse, VAEs with blurry samples.
- Diffusion models replace one hard optimisation with many easy ones. The objective is a simple regression onto the added noise, training is stable, and sample quality is state of the art.
- The price is **iterative sampling**: hundreds of network evaluations per image. Most of this lecture is about paying that price down, by moving to a latent space, by better samplers, and by distillation.



I. Aristimuño, An Introduction to Diffusion Models and Stable Diffusion, Marvik blog, 2023

Quick Overview

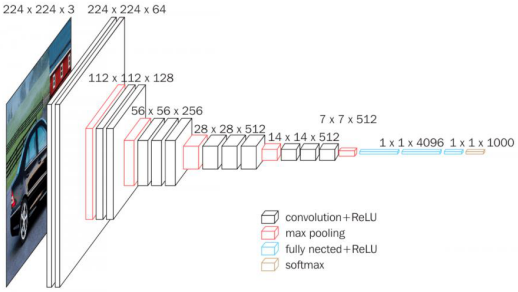
- **Forward:** Clean $\rightarrow$ Noise (additive Gaussian steps)
- **Reverse:** Learn **U-Net** to remove noise step by step
- **Training:** Model predicts the noise added
- **Sampling:** Roll back from pure noise to a data sample

Stable Diffusion Core Backbone:

Attention-U-Net Architecture

Refresher: Convolutional Neural Network

- CNN parametrizes a function over images.
- **Motivation:**
  - Features are translationally invariant.
  - Extract features at different scales / abstraction levels.
- **Key modules:**
  - Convolution
  - Downsampling (max-pool)
- Features of larger scale (larger receptive field) $\Rightarrow$ higher abstraction level.

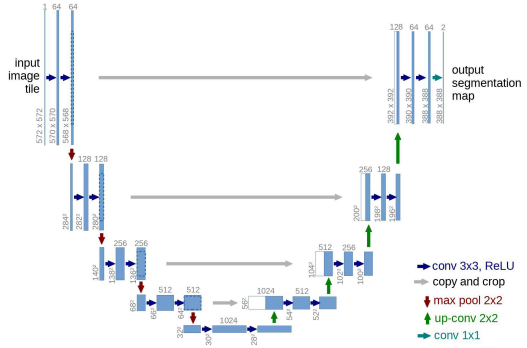


Binxu Wang & John Vastola, Stable Diffusion: A Tutorial (Harvard ML from Scratch), 2022.

CNN + inverted CNN $\Rightarrow$ U-Net

- An inverted CNN (generator) can generate images.
- CNN + inverted CNN can model an image-to-image function.
- **Down sampling:** convolution.
- **Up sampling:** transposed convolution.

U-Net: A Natural Architecture for Image-to-Image Functions



Binxu Wang & John Vastola, Stable Diffusion: A Tutorial (Harvard ML from Scratch), 2022.

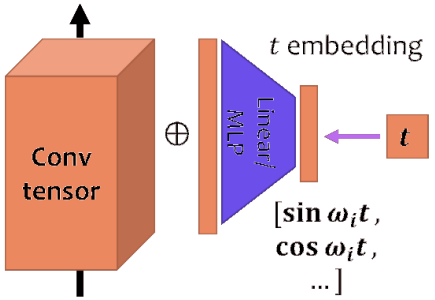
Note: Add Time Dependency

The score function is time-dependent.

Target: $s(x,t) = \nabla_x \log p_t(x)$, the score of the noised marginal at time t. It is the same object the noise predictor learns, up to a scale: $s(x,t) \approx -\epsilon_\theta(x,t)/\sigma_t$.

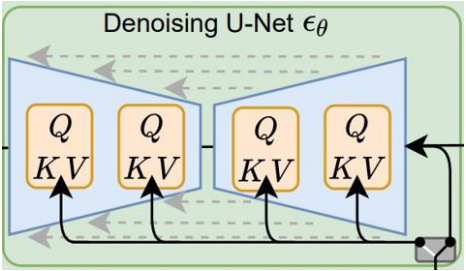
Add time dependency

- Assume time dependency is spatially homogeneous.
- Add one scalar value per channel $f(t)$.
- Parametrize $f(t)$ by MLP/linear of Fourier basis.
  - Fourier features: $[\sin \omega_i t, \cos \omega_i t, \ldots]$



Binxu Wang & John Vastola, Stable Diffusion: A Tutorial (Harvard ML from Scratch), 2022.

U-Net in Stable Diffusion

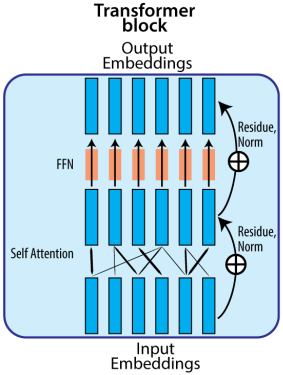


Binxu Wang & John Vastola, Stable Diffusion: A Tutorial (Harvard ML from Scratch), 2022.

- Input convolution: Projects 4-channel input $\rightarrow$ 320 channels using a 2D convolution.
- Time embedding: inject time information into the network.
- Downsampling: sometimes interleaved with cross-attention.
- Mid block: applies additional residual and (cross-)attention operations.
- Upsampling, normalization, activation, output convolution.

How to Understand Prompts: Language Model Refresher

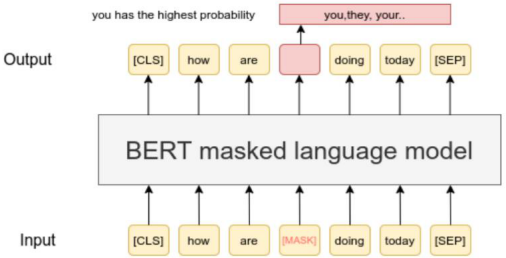
- Meaning of a word depends on context, not always the same.
- Example: "I *book* a ticket to buy that *book*."
- Word vectors should depend on context.
- Transformers let each word absorb influence from other words to become contextualized.



Binxu Wang & John Vastola, Stable Diffusion: A Tutorial (Harvard ML from Scratch), 2022.

Learning Word Vectors: GPT, BERT & CLIP

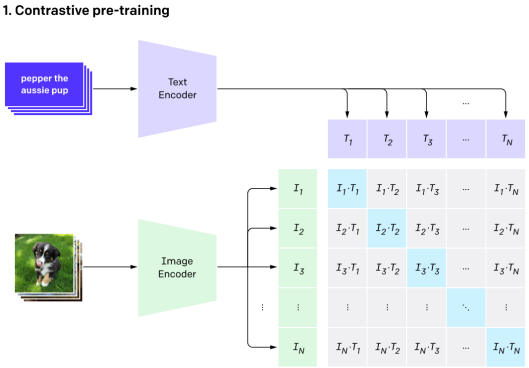
- Self-supervised learning of word representations.
- Predict missing / next words in a sentence (BERT, GPT).
- Contrastive learning: match image and text (CLIP).
- Downstream classifiers can decode part of speech, sentiment, ...



Binxu Wang & John Vastola, Stable Diffusion: A Tutorial (Harvard ML from Scratch), 2022.

Joint Representation for Vision and Language: CLIP

- Learn a joint encoding space for text captions and images.
- Maximize representation similarity between an image and its caption.
- Minimize similarity for other pairs.



Adapted from: Radford et al., CLIP, arXiv:2103.00020

Choice of Text Encoding

- Encoder in Stable Diffusion v1: frozen, pre-trained CLIP ViT-L/14 text encoder. Later versions differ: SD 2 uses OpenCLIP ViT-H/14, SDXL two encoders, SD 3 adds T5-XXL.
- Word vectors can also be randomly initialized and learned online.
- **Other conditional signals:**
  - Object categories (e.g. Shark, Trout): one vector per class.
  - Face attributes (e.g. {female, blonde hair, with glasses}): one vector per attribute.
- Time to be creative!

Stable Diffusion:

A Latent Diffusion Model (LDM)

Perceptual vs. Semantic Compression

Pixel-space diffusion is redundant: the U-Net spends capacity on imperceptible high-frequency detail. **Stable Diffusion** (Rombach et al., 2022) decouples:

1. **Perceptual compression (VAE):** $\mathcal{E} : \mathbf{x} \mapsto \mathbf{z}$; remove imperceptible spatial detail, preserve layout.
2. **Semantic generation (LDM):** run diffusion on $\mathbf{z}$; lower memory, faster training.
3. **Decode:** $\mathcal{D}(\mathbf{z}) \mapsto \mathbf{x}$; project back to pixels.

Typical spatial downsampling: $8\times$ (e.g., $512 \times 512 \times 3$ pixels $\rightarrow 64 \times 64 \times 4$ latents, a $\sim 48\times$ reduction in dimensions).

Cross-Attention: Injecting Text into the U-Net

Pipeline: CLIP text encoder $\rightarrow$ token embeddings $\mathbf{c} \in \mathbb{R}^{L \times d}$.

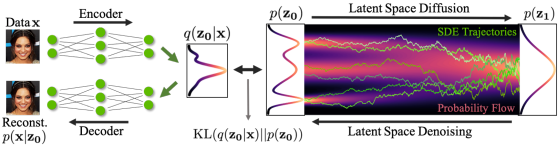
- **Self-attention:** Q, K, V all come from spatial latent features; provides global visual coherence.
- **Cross-attention:** Q from spatial features; K, V from the text embeddings; aligns latent spatial positions with prompt tokens.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Scaling by $\sqrt{d_k}$ prevents softmax saturation in high dimensions.

Token length $L \leq 77$ (CLIP); multi-head attention throughout the U-Net blocks.

What is a Latent Space?



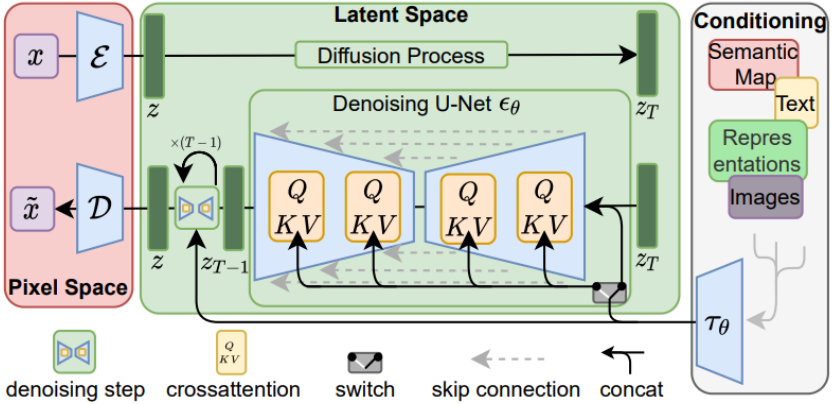
Vahdat, Kreis & Kautz, Score-based Generative Modeling in Latent Space, NeurIPS 2021, Fig. 1

- A **latent space** is an abstract, compressed space where high-dimensional data (like images) is represented in a lower-dimensional form.
- Think of it as a hidden layer of meaning: each point in this space represents a potential image or concept.
- In classical models like GANs or VAEs, the latent space is directly sampled from (e.g., a vector $\mathbf{z}$), and the generator turns it into an image.
- **Latent** space = "Hidden space of ideas or features"
- Note that even a pixel-space diffusion model has a latent code: the initial noise vector. Changing it gives a different image, and interpolating it morphs between images.

Why Latent Diffusion?

- **Traditional pixel-space diffusion models are computationally expensive**: they operate directly on high-dimensional images, making both training and inference costly.
- **Latent diffusion reduces computational cost** by performing the diffusion process in a compressed, lower-dimensional latent space, while still maintaining high-quality image generation.
- This efficiency enables more complex and practical applications, such as text-guided generation and flexible image editing.
- **How is this achieved?** Instead of working in pixel space, we learn a compressed latent space using a Variational Autoencoder (VAE) with very low KL divergence weight (e.g., 10^{-6}). This yields a compact representation suitable for diffusion.
- This mirrors the two-stage recipe of VQGAN: compress to a discrete or continuous code with an autoencoder, then model the distribution over that code.

Latent Diffusion Models: Visual Overview



Source: Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022, Fig. 3.

Stable Diffusion:

Guidance and Sampling

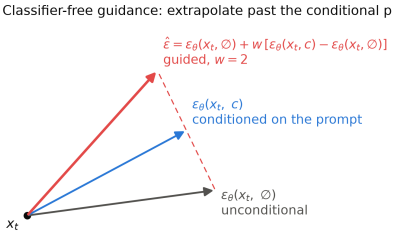
Classifier-Free Guidance (CFG)

- Conditioning alone follows the prompt only loosely; every real system amplifies it with **classifier-free guidance** (Ho & Salimans, 2022).
- **Training trick:** replace the text conditioning with a null token for a random fraction of training *examples* ($\sim$10%), so one network learns both a **conditional** and an **unconditional** noise prediction.
- **At sampling time**, run both and **extrapolate**:

$$\hat{\epsilon} = \epsilon_\theta(x_t, \varnothing) + w\,[\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \varnothing)]$$

- The guidance scale w trades **prompt fidelity** against **diversity**: $w = 1$ is plain conditional sampling; SD v1 defaults to $w \approx 7.5$; too high causes oversaturated, artifact-prone images.

CFG, Geometrically



- Each denoising step pushes the sample **past** the conditional prediction, away from the unconditional one: whatever the prompt adds gets amplified.
- **Negative prompts** reuse the same mechanism: replace the unconditional branch with an embedding of what you do *not* want, and the extrapolation pushes away from it.

Samplers: From 1000 Steps to 20

- DDPM training uses ∼1000 noise levels, but sampling does not have to visit all of them.
- **DDIM** (Song et al., 2021): a deterministic sampler over the same trained model; skips steps and enables consistent latent-space edits. Typically 20–50 steps.
- **ODE-solver view:** sampling is integrating a differential equation, so better integrators reach good quality in fewer steps. Euler is first order but works well on the probability-flow ODE with a good noise schedule; DPM-Solver++ and Heun are higher order.
- **Distillation** goes further, to 1–4 steps: latent consistency models (LCM), adversarial diffusion distillation (SDXL Turbo), and SD 3.5 **Large Turbo** all distill a many-step teacher into a few-step student.
- Practical takeaway: the model, the sampler, and the step count are **independent choices**; most quality-speed tuning happens here.

Stable Diffusion:

Customizing Stable Diffusion

Personalization: Textual Inversion, DreamBooth, LoRA

- Open weights made SD the platform people **fine-tune**; three techniques cover most of it:
- **Textual Inversion** (Gal et al., 2023): learn a **new token embedding** for a concept from a few images; the rest of the model stays frozen ($\sim$kilobytes per concept).
- **DreamBooth** (Ruiz et al., 2023): fine-tune the model itself to bind a **rare token** to a specific subject (your pet, a product), with a prior-preservation loss against forgetting the class.
- **LoRA** (Hu et al., 2022): train **low-rank adapters** on the attention weights instead of the full model; megabytes instead of gigabytes, composable, and cheap to share. The de facto standard for style and character adaptation.

DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation

Nataniel Ruiz Yuanzhen Li Varun Jampani Yael Pritch Michael Rubinstein Kfir Aberman

Google Research

It's like a photo booth, but once the subject is captured, it can be synthesized wherever your dreams take you…

Source: Ruiz et al., "DreamBooth", CVPR 2023.

DreamBooth (cont.)

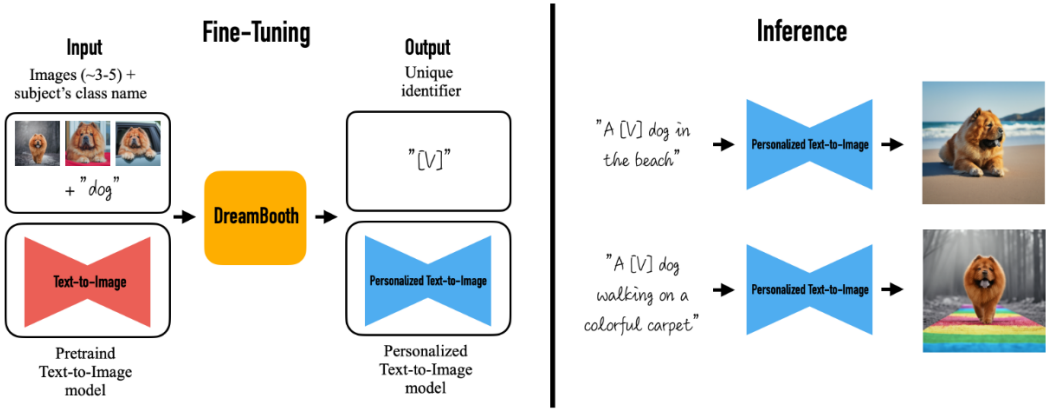
What is DreamBooth?

- A method for fine-tuning large diffusion models on a small number of images.
- Developed by Google Research; published at CVPR 2023.
- Allows personalization of generative models with minimal data.

Key Concepts

- **Personalization:** Adapts a pre-trained model to generate images of specific subjects (e.g., pets, people).
- **Fine-tuning:** Uses a small set of images to adjust the model's parameters.
- **Text Conditioning:** Uses text prompts to guide image generation.

DreamBooth (cont.)

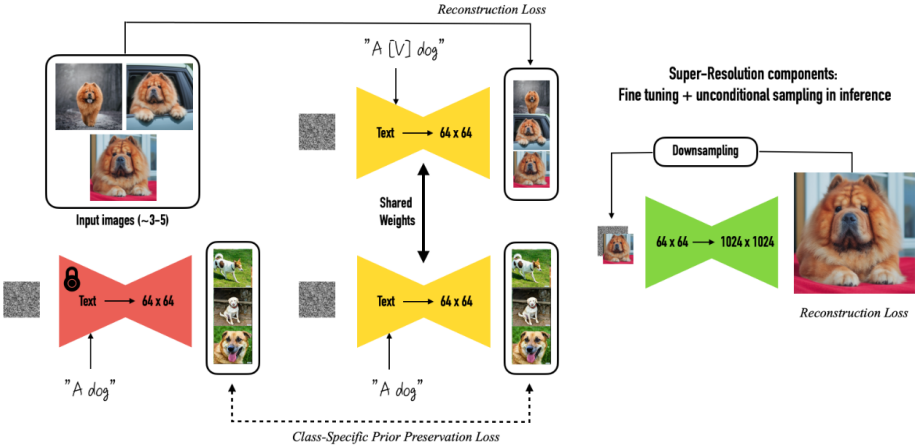


Source: Ruiz et al., "DreamBooth", CVPR 2023.

How Does DreamBooth Personalize Models?

- DreamBooth teaches a model to recognize a new subject using just a few photos.
- After training, the model can create new images of that subject in different places or situations.
- This is called "few-shot" learning: only a few examples are needed.

DreamBooth (cont.)



Source: Ruiz et al., "DreamBooth", CVPR 2023.

DreamBooth (cont.)

How Do We Tell the Model About the New Subject?

- We bind the subject to a *rare token*: an existing vocabulary token that the model has almost never seen in training.
- For example: "a photo of `sks dog` at the beach" (`sks` is the community convention popularised by the `diffusers` examples, not a token from the paper).
- The model learns to link this rare-token to the subject in your photos.

Why Use Rare-tokens?

- Rare tokens are existing vocabulary tokens that appear very infrequently, so they carry a weak prior and are easy to rebind.
- This avoids mixing up your subject with things the model already knows.

How Does DreamBooth Avoid Forgetting?

- If we only train on your subject, the model might forget how to make other images.
- To prevent this, we also show it regular class prompts (like "a photo of a dog").
- We use a special loss to balance learning the new subject and keeping old knowledge:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{subject}} + \lambda \mathcal{L}_{\text{prior}}$$

where:

- $\mathcal{L}_{\text{subject}}$ = loss for your subject (rare-token prompt)
- $\mathcal{L}_{\text{prior}}$ = loss for general class (class prompt)
- λ = weight to balance the two

DreamBooth: Results

Source: Ruiz et al., "DreamBooth", CVPR 2023.

DreamBooth: Results (cont.)

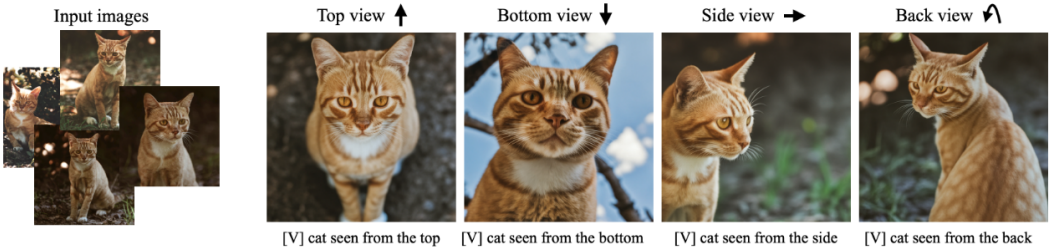
Art Rendition

Original artistic renditions of our subject dog in the style of famous painters. We remark that many of the generated poses were not seen in the training set, such as the Van Gogh and Warhol rendition. We also note that some renditions seem to have novel composition and faithfully imitate the style of the painter - even suggesting some sort of creativity (extrapolation given previous knowledge).

Source: Ruiz et al., "DreamBooth", CVPR 2023.

Text-Guided View Synthesis

Our technique can synthesized images with specified viewpoints for a subject cat (left to right: top, bottom, side and back views). Note that the generated poses are different from the input poses, and the background changes in a realistic manner given a pose change. We also highlight the preservation of complex fur patterns on the subject cat's forehead.

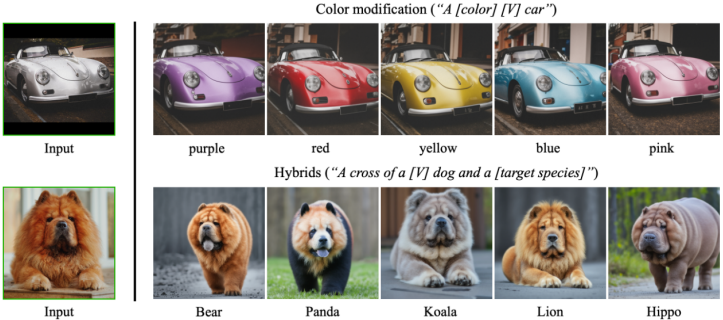


Source: Ruiz et al., "DreamBooth", CVPR 2023.

DreamBooth: Results (cont.)

Property Modification

We show color modifications in the first row (using prompts ``a [color] [V] car''), and crosses between a specific dog and different animals in the second row (using prompts ``a cross of a [V] dog and a [target species]''). We highlight the fact that our method preserves unique visual features that give the subject its identity or essence, while performing the required property modification.



Source: Ruiz et al., “DreamBooth”, CVPR 2023.

DreamBooth: Results (cont.)

Accessorization

Outfitting a dog with accessories. The identity of the subject is preserved and many different outfits or accessories can be applied to the dog given a prompt of type *"a [V] dog wearing a police/chef/witch outfit"*. We observe a realistic interaction between the subject dog and the outfits or accessories, as well as a large variety of possible options.

Source: Ruiz et al., "DreamBooth", CVPR 2023.

Low-Rank Adaptation (LoRA)

LoRA: Low-Rank Adaptation of Large Language Models

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu

Source: Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022.

LoRA Overview

Low-Rank Adaptation (LoRA) is a technique for efficient fine-tuning of large neural networks. Instead of updating all parameters, LoRA injects trainable low-rank matrices into each layer, significantly reducing the number of trainable parameters.

Key Idea: Decompose the weight update ΔW as a product of two low-rank matrices:

$$\Delta W = BA \tag{1}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, with $r \ll \min(d, k)$. Following Hu et al., A is the down-projection (random Gaussian init) and B the up-projection (initialised to zero), so $\Delta W = 0$ at the start of training.

Low-Rank Adaptation (LoRA) (cont.)

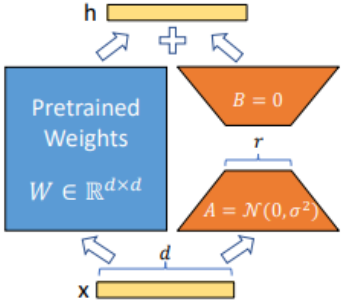


Figure 1: Our reparametrization. We only train A and B.

Source: Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022.

LoRA in Linear Layers

Consider a linear layer with weight $W_0 \in \mathbb{R}^{d \times k}$. In LoRA, the forward pass is modified as:

$$y = W_0 x + \frac{\alpha}{r} BAx \tag{2}$$

where α is a constant scaling factor and only A and B are updated during fine-tuning. Dividing by r keeps the size of the update roughly independent of the rank, so r can be changed without retuning the learning rate.

Parameter Efficiency: The number of trainable parameters is reduced from $d \times k$ to $r \times (d + k)$.

LoRA Training Objective

The training objective remains the same as standard fine-tuning, e.g., minimizing a loss function $\mathcal{L}$:

$$\min_{A,B} \mathcal{L}(\hat{y}, y) \tag{3}$$

where $\hat{y}$ is the model output and y is the ground truth.

Regularization: The rank bound is structural, not a penalty: $\Delta W = BA$ has rank at most r by construction. Weight decay or dropout on the adapter branch can still be added to limit overfitting.

LoRA in Attention Mechanisms

LoRA is commonly applied to the query and value projection matrices in attention layers:

$$W_q' = W_q + \Delta W_q = W_q + \frac{\alpha}{r} B_q A_q \tag{4}$$

$$W_v' = W_v + \Delta W_v = W_v + \frac{\alpha}{r} B_v A_v \tag{5}$$

where B_q, A_q, B_v, A_v are the low-rank factors for the query and value projections, respectively.

Low-Rank Adaptation (LoRA) (cont.)

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB_{base} (FT)*	125.0M	**87.6**	94.8	90.2	**63.6**	92.8	**91.9**	78.7	91.2	86.4
RoB_{base} (BitFit)*	0.1M	84.7	93.7	**92.7**	62.0	91.8	84.0	81.5	90.8	85.2
RoB_{base} (Adpt^{D})*	0.3M	$87.1_{\pm.0}$	$94.2_{\pm.1}$	$88.5_{\pm1.1}$	$60.8_{\pm.4}$	$93.1_{\pm.1}$	$90.2_{\pm.0}$	$71.5_{\pm2.7}$	$89.7_{\pm.3}$	84.4
RoB_{base} (Adpt^{D})*	0.9M	$87.3_{\pm.1}$	$94.7_{\pm.3}$	$88.4_{\pm.1}$	$62.6_{\pm.9}$	$93.0_{\pm.2}$	$90.6_{\pm.0}$	$75.9_{\pm2.2}$	$90.3_{\pm.1}$	85.4
RoB_{base} (LoRA)	0.3M	$87.5_{\pm.3}$	$\mathbf{95.1}_{\pm.2}$	$89.7_{\pm.7}$	$63.4_{\pm1.2}$	$\mathbf{93.3}_{\pm.3}$	$90.8_{\pm.1}$	$\mathbf{86.6}_{\pm.7}$	$\mathbf{91.5}_{\pm.2}$	**87.2**
$\text{RoB}_{\text{large}}$ (FT)*	355.0M	90.2	**96.4**	**90.9**	68.0	94.7	**92.2**	86.6	92.4	88.9
$\text{RoB}_{\text{large}}$ (LoRA)	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.9}_{\pm1.2}$	$\mathbf{68.2}_{\pm1.9}$	$\mathbf{94.9}_{\pm.3}$	$91.6_{\pm.1}$	$\mathbf{87.4}_{\pm2.5}$	$\mathbf{92.6}_{\pm.2}$	**89.0**
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	3.0M	$90.2_{\pm.3}$	$96.1_{\pm.3}$	$90.2_{\pm.7}$	$\mathbf{68.3}_{\pm1.0}$	$\mathbf{94.8}_{\pm.2}$	$\mathbf{91.9}_{\pm.1}$	$83.8_{\pm2.9}$	$92.1_{\pm.7}$	88.4
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	0.8M	$\mathbf{90.5}_{\pm.3}$	$\mathbf{96.6}_{\pm.2}$	$89.7_{\pm1.2}$	$67.8_{\pm2.5}$	$\mathbf{94.8}_{\pm.3}$	$91.7_{\pm.2}$	$80.1_{\pm2.9}$	$91.9_{\pm.4}$	87.9
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	6.0M	$89.9_{\pm.5}$	$96.2_{\pm.3}$	$88.7_{\pm2.9}$	$66.5_{\pm4.4}$	$94.7_{\pm.2}$	$92.1_{\pm.1}$	$83.4_{\pm1.1}$	$91.0_{\pm1.7}$	87.8
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	0.8M	$90.3_{\pm.3}$	$96.3_{\pm.5}$	$87.7_{\pm1.7}$	$66.3_{\pm2.0}$	$94.7_{\pm.2}$	$91.5_{\pm.1}$	$72.9_{\pm2.9}$	$91.5_{\pm.5}$	86.4
$\text{RoB}_{\text{large}}$ (LoRA)†	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.2}_{\pm1.0}$	$68.2_{\pm1.9}$	$\mathbf{94.8}_{\pm.3}$	$91.6_{\pm.2}$	$\mathbf{85.2}_{\pm1.1}$	$\mathbf{92.3}_{\pm.5}$	**88.6**
DeB_{XXL} (FT)*	1500.0M	91.8	**97.2**	92.0	72.0	**96.0**	92.7	93.9	92.9	91.1
DeB_{XXL} (LoRA)	4.7M	$\mathbf{91.9}_{\pm.2}$	$96.9_{\pm.2}$	$\mathbf{92.6}_{\pm.6}$	$\mathbf{72.4}_{\pm1.1}$	$\mathbf{96.0}_{\pm.1}$	$\mathbf{92.9}_{\pm.1}$	$\mathbf{94.9}_{\pm.4}$	$\mathbf{93.0}_{\pm.2}$	**91.3**

Table 2: $\text{RoBERTa}_{\text{base}}$, $\text{RoBERTa}_{\text{large}}$, and $\text{DeBERTa}_{\text{XXL}}$ with different adaptation methods on the GLUE benchmark. We report the overall (matched and mismatched) accuracy for MNLI, Matthew's correlation for CoLA, Pearson correlation for STS-B, and accuracy for other tasks. Higher is better for all metrics. * indicates numbers published in prior works. † indicates runs configured in a setup similar to Houlsby et al. (2019) for a fair comparison.

Source: Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022.

Low-Rank Adaptation (LoRA) (cont.)

Model & Method	# Trainable Parameters	E2E NLG Challenge BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M ($\text{Adapter}^{\text{L}}$)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M ($\text{Adapter}^{\text{L}}$)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M ($\text{Adapter}^{\text{H}}$)	11.09M	$67.3_{\pm.6}$	$8.50_{\pm.07}$	$46.0_{\pm.2}$	$70.7_{\pm.2}$	$2.44_{\pm.01}$
GPT-2 M (FT^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	$\mathbf{70.4}_{\pm.1}$	$\mathbf{8.85}_{\pm.02}$	$\mathbf{46.8}_{\pm.2}$	$\mathbf{71.8}_{\pm.1}$	$\mathbf{2.53}_{\pm.02}$
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L ($\text{Adapter}^{\text{L}}$)	0.88M	$69.1_{\pm.1}$	$8.68_{\pm.03}$	$46.3_{\pm.0}$	$71.4_{\pm.2}$	$\mathbf{2.49}_{\pm.0}$
GPT-2 L ($\text{Adapter}^{\text{L}}$)	23.00M	$68.9_{\pm.3}$	$8.70_{\pm.04}$	$46.1_{\pm.1}$	$71.3_{\pm.2}$	$2.45_{\pm.02}$
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	$\mathbf{70.4}_{\pm.1}$	$\mathbf{8.89}_{\pm.02}$	$\mathbf{46.8}_{\pm.2}$	$\mathbf{72.0}_{\pm.2}$	$2.47_{\pm.02}$

Table 3: GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters. Confidence intervals are shown for experiments we ran. * indicates numbers published in prior works.

Source: Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022.

Low-Rank Adaptation (LoRA) (cont.)

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($\text{Adapter}^{\text{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($\text{Adapter}^{\text{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1$ for the three metrics.

Source: Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022.

Low-Rank Adaptation (LoRA) (cont.)

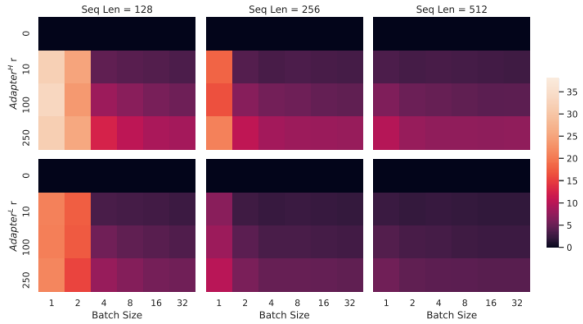


Figure 5: Percentage slow-down of inference latency compared to the no-adapter ($r = 0$) baseline. The top row shows the result for Adapter$^{\text{H}}$ and the bottom row Adapter$^{\text{L}}$. Larger batch size and sequence length help to mitigate the latency, but the slow-down can be as high as over 30% in an online, short-sequence-length scenario. We tweak the colormap for better visibility.

Source: Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022.

Spatial Control: ControlNet

- Text describes *what*; it is poor at *where*. **ControlNet** (Zhang et al., 2023) adds spatial conditioning: edge maps, human pose, depth, sketches, segmentation.
- Mechanism: clone the U-Net encoder into a **trainable copy** that ingests the condition image, keep the original weights **frozen**, and connect the copy through **zero-initialized convolutions** so training starts from a no-op and cannot damage the base model.
- Result: precise layout control with modest paired data, reusing everything the base model knows.
- Together with LoRA and inpainting, this is why an open 2022 model still anchors production pipelines today.

Stable Diffusion:

Applications and Lineage

Application: Stable Video Diffusion

- Initialize from Stable Diffusion.
- Add temporal layers (e.g., 1D Conv, temporal attention).
- Finetune on video data.
- Data pipeline:
  - Scrape **unlabeled** videos from the internet.
  - Use image and video captioners to generate synthetic text labels.
  - Apply aggressive data filtering using several heuristics (CLIP score, optical flow score, aesthetic scores, OCR, etc.).

Application: Stable Video Diffusion (cont.)

Source: Blattmann et al., "Stable Video Diffusion", 2023; sample tile from the release repository Stability-AI/generative-models. Animated version.

Lineage: From GLIDE to Stable Diffusion

- **GLIDE** (Nichol et al., 2021): a 3.5B-parameter text-conditional diffusion model. It compared CLIP guidance against classifier-free guidance and found human raters preferred the latter; fine-tuning it gave text-driven inpainting.
- **DALL·E 2** (Ramesh et al., 2022): two stages, a **prior** mapping a caption to a CLIP image embedding and a **decoder** diffusing an image from it. The shared CLIP space is what buys image variations and zero-shot edits.
- **Imagen** (Saharia et al., 2022): a **frozen**, text-only language model (T5-XXL) as the text encoder. Its headline finding is that scaling the language model improves fidelity and image-text alignment far more than scaling the image diffusion model.
- **Stable Diffusion** (Rombach et al., 2022): all three of the above denoise at pixel resolution and stack upsamplers on top. SD instead moves the diffusion into a **VAE latent space**, which is what made an openly released model that runs on a single consumer GPU possible.

GLIDE arXiv:2112.10741; DALL·E 2 arXiv:2204.06125; Imagen arXiv:2205.11487; LDM arXiv:2112.10752.

Application: Stable Diffusion 2.0

What changed vs. Stable Diffusion v1?

- **New text encoder:** OpenCLIP (LAION + Stability AI) replaces CLIP ViT-L/14, giving stronger language understanding and image quality.
- **Native resolutions:** text-to-image at **512×512** and **768×768**.
- **Upscaler diffusion:** ×4 super-resolution (e.g. 128→512).
- **depth2img:** depth-guided image-to-image; structure-preserving synthesis from inferred depth + text.
- **Updated inpainting:** text-guided inpainting fine-tuned on the Stable Diffusion 2.0 base model.

Lopez, "Stable Diffusion 2.0 Release," Stability AI (Nov 2022).

Stable Diffusion 2.0: Upscaler Diffusion

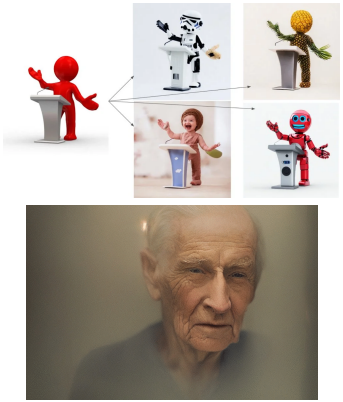
- Dedicated **upscaler diffusion** model enhances resolution by $\times 4$.
- Example: $\mathbf{128}\times\mathbf{128}$ low-res sample $\rightarrow$ $\mathbf{512}\times\mathbf{512}$ output.

Left: 128×128. Right: 512×512 upscaled.

Stability AI, Stable Diffusion 2.0 Release.

Stable Diffusion 2.0: depth2img

- **depth2img** infers depth from an input image, then generates new images conditioned on **text + depth**.
- Enables structure-preserving image-to-image and shape-conditional synthesis.
- Can produce radically different appearances while preserving spatial coherence and depth.



Input → variants (top); depth coherence preserved (bottom).

Stability AI, Stable Diffusion 2.0 Release.

Stable Diffusion 2.0: Inpainting

Text-guided inpainting model fine-tuned on the SD 2.0 text-to-image base.

Stability AI, Stable Diffusion 2.0 Release.

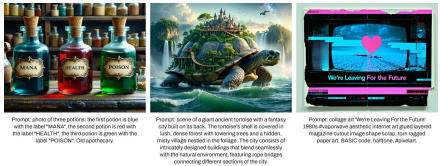
Application: SDXL

- **SDXL** (Podell et al., 2023): the scaled-up successor to SD 2; for a long time the most-used open text-to-image model.
- A 2.6B-parameter U-Net (about 3× the 860M of SD 1/2) with **two text encoders** used together (OpenCLIP ViT-bigG + CLIP ViT-L).
- Native **1024×1024** generation, plus micro-conditioning on image size and crop so the model learns from mixed-resolution data without artifacts.
- Optional **refiner** model: a second latent-diffusion stage specialized in high-frequency detail.

Inside Stable Diffusion 3: MM-DiT and Rectified Flow

- SD 3 changes both the backbone and the training objective (Esser et al., 2024).
- **Backbone:** the U-Net is replaced by a **multimodal diffusion transformer (MM-DiT)**, following the DiT line (Peebles & Xie, 2023): text and image tokens flow through a joint transformer with separate weights per modality.
- **Objective: rectified flow**, i.e. flow matching on straight noise-data paths, instead of the DDPM objective; the paper's ablations found it superior at scale. (Flow matching was covered earlier in the course.)
- **Text encoders:** two CLIP encoders plus **T5-XXL** for long, compositional prompts and better in-image text.
- **FLUX.1** from Black Forest Labs (2024), a lab founded by members of the team behind VQGAN, Latent Diffusion and Stable Diffusion, continues this rectified-flow-transformer lineage as the strongest open release of its generation.

Stable Diffusion 3.5: Model Family and Hardware Footprint



Prompt: photo of three potions: the first potion is blue with the label "MANA", the second potion is red with the label "HEALTH", the third potion is green with the label "POISON". Old apothecary.

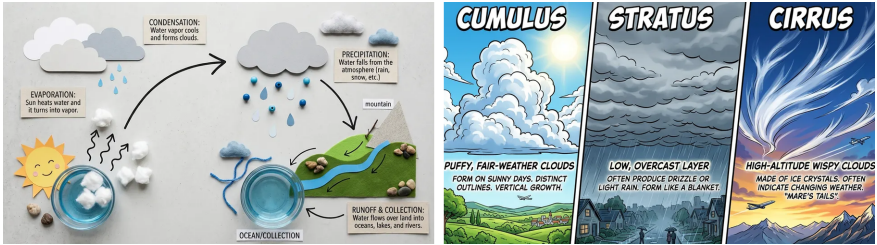
Prompt: scene of a giant ancient tortoise with a fantasy city built on its back. The tortoise's shell is covered in lush, dense forest with towering trees and a hidden, misty village nestled in the foliage. The city consists of intricately designed buildings that blend seamlessly with the natural environment, featuring rope bridges connecting different sections of the city.

Prompt: collage art 'We're Leaving For the Future' 1980s #vaporwave aesthetic internet art glued layered magazine cutout image shape soap, torn ragged paper art, BASIC code, halftone, #pixelart.

Source: Stability AI, "Introducing Stable Diffusion 3.5" (Oct 2024).

- **Customizability:** Large (8.1B), Large Turbo (8.1B) and Medium (2.5B) models.
- **Efficient Performance:** Optimized to run on standard consumer hardware; Medium and Large Turbo require no specialized GPUs.
- **Highly Accessible:** Medium needs 9.9 GB of VRAM excluding the text encoders, making advanced diffusion imaging feasible on most consumer GPUs.

Application: Nano Banana 2 – Google DeepMind



Examples: water-cycle infographic and cloud-type comparison.

- A closed-weight **Gemini** image model rather than a latent diffusion model, included here for contrast. It builds on Google's Gemini Image line: Nano Banana (2025) → Nano Banana Pro → Nano Banana 2.
- Production-ready output: multiple aspect ratios and resolutions from 512px to 4K.

Raisinghani, "Nano Banana 2: Combining Pro capabilities with lightning-fast speed," The Keyword, Google blog (Feb 2026).

Try It Yourself!

Text to Image generation with stable diffusion.
https://huggingface.co/spaces/stabilityai/stable-diffusion-3.5-large

Esser et al., "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis" (Stable Diffusion 3), ICML 2024

Stable Diffusion:

Summary & Limitations

From diffusion to Stable Diffusion

- Diffusion models learn to reverse a gradual noising process; sampling remains iterative but training is stable and scalable.
- **Latent Diffusion (LDM):** denoise in a VAE-compressed latent space ($\sim 8\times$ downsampling) rather than full pixel space: lower memory and faster training/inference.

Core architecture

- **Attention U-Net:** encoder–decoder with skip connections; downsampling / upsampling for multiscale features.
- **Time conditioning:** Fourier-feature time embeddings added per channel.
- **Text conditioning:** CLIP (ViT-L/14 in SD v1) maps prompts to token embeddings; cross-attention injects language into the U-Net.

Summary (cont.)

Using and steering the model

- **Classifier-free guidance:** extrapolate past the conditional prediction with scale w; negative prompts reuse the same mechanism.
- **Samplers:** DDIM and higher-order ODE solvers cut 1000 steps to 20–50; distillation (LCM, Turbo) reaches 1–4.
- **Customization:** Textual Inversion, DreamBooth, and LoRA personalize; ControlNet adds spatial control.

Summary (cont.)

Model evolution & applications

- **SD 2.0:** OpenCLIP text encoder; native 512/768 T2I; upscaler ($\times 4$), depth2img, and updated inpainting.
- **SDXL:** 2.6B U-Net ($\sim$3.5B total), dual text encoders, native 1024^2, optional refiner.
- **SD 3 / 3.5:** MM-DiT transformer backbone trained with rectified flow; Large and Large Turbo are both 8.1B, Medium is 2.5B and is the one that fits comfortably on a consumer GPU; FLUX.1 continues the lineage.
- Around it: the pixel-cascade predecessors it replaced (GLIDE, DALL·E 2, Imagen), video built on its backbone (Stable Video Diffusion), and newer closed multimodal systems (e.g. Nano Banana 2).

Limitations

- **Slow sampling:** many denoising steps (partially addressed by distilled models such as SD 3.5 Large Turbo).
- **Data & compute:** high-quality open models still require large-scale curated data and substantial GPU resources to train.
- **Prompt sensitivity:** output quality and adherence depend on prompt wording; base models may vary across seeds.
- **Distribution gaps:** rare concepts, fine details, and compositional reasoning remain challenging.
- **Safety & misuse:** text-to-image systems raise concerns around deepfakes, copyright, and misinformation.

References

Tutorials and Surveys

- Binxu Wang & John Vastola (2022). Stable Diffusion: A Tutorial (Harvard ML from Scratch): U-Net, time embedding, CLIP conditioning.
- Kreis, K., Gao, R., & Vahdat, A.: CVPR 2022 Tutorial: Diffusion Models
- Rogge, N., & Rasul, K.: The Annotated Diffusion Model
- Lilian Weng: What are Diffusion Models?
- Yang Song: Score-Based Generative Modeling: A Brief Overview
- Binxu Wang: DiffusionFromScratch: companion code that rebuilds Stable Diffusion in a single script.

References (cont.)

Foundational Diffusion Models

- Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models
- Sohl-Dickstein, J., et al. (2015). Deep Unsupervised Learning using Nonequilibrium Thermodynamics
- Song, Y., & Ermon, S. (2019). Generative Modeling by Estimating Gradients of the Data Distribution
- Nichol, A., & Dhariwal, P. (2021). Improved Denoising Diffusion Probabilistic Models

Stable Diffusion: Core Papers

- Rombach, R., et al. (2022). High-Resolution Image Synthesis with Latent Diffusion Models: Stable Diffusion / LDM (VAE + U-Net + conditioning).
- CompVis (2022). CompVis/stable-diffusion: the original open code and weight release of the paper above; source of the text-to-image sample strip.
- Radford, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP): text encoder and cross-attention conditioning.

References (cont.)

Guidance, Sampling, and Customization

- Ho, J., & Salimans, T. (2022). Classifier-Free Diffusion Guidance
- Song, J., Meng, C., & Ermon, S. (2021). Denoising Diffusion Implicit Models (DDIM), ICLR 2021.
- Lu, C., et al. (2022). DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models
- Luo, S., et al. (2023). Latent Consistency Models
- Sauer, A., et al. (2023). Adversarial Diffusion Distillation (SDXL Turbo)
- Gal, R., et al. (2023). An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion, ICLR 2023.
- Ruiz, N., et al. (2023). DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation, CVPR 2023.
- Hu, E., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models, ICLR 2022.

References (cont.)

- Zhang, L., Rao, A., & Agrawala, M. (2023). Adding Conditional Control to Text-to-Image Diffusion Models (ControlNet), ICCV 2023.

References (cont.)

Applications and Model Releases

- Nichol, A., et al. (2021). GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models
- Ramesh, A., et al. (2022). Hierarchical Text-Conditional Image Generation with CLIP Latents (DALL·E 2)
- Saharia, C., et al. (2022). Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding (Imagen)
- Blattmann, A., et al. (2023). Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets
- Podell, D., et al. (2024). SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis, ICLR 2024.
- Peebles, W., & Xie, S. (2023). Scalable Diffusion Models with Transformers (DiT), ICCV 2023.

References (cont.)

- Esser, P., et al. (2024). Scaling Rectified Flow Transformers for High-Resolution Image Synthesis (Stable Diffusion 3), ICML 2024.
- Black Forest Labs (2024). Announcing Black Forest Labs (FLUX.1)
- Lopez, J. (2022). Stable Diffusion 2.0 Release: OpenCLIP, upscaler, depth2img, inpainting (Stability AI blog).
- Stability AI (2024). Introducing Stable Diffusion 3.5: SD 3.5 Large / Large Turbo / Medium (Stability AI blog).
- Raisinghani, N. (2026). Nano Banana 2: Combining Pro capabilities with lightning-fast speed (The Keyword, Google blog).